Day08_字符串

什么是字符串

在C中，字符串，就是一个特殊的字符数组。
以 \0结尾。

C里面的字符串设计不是特别好。
问题1： 字符串本身无法存储 \0
问题2： 每一次看字符串的长度，都要遍历一遍字符数组

在后期的编程语言设计中，基本都会采用
字符数组+额外长度信息来 整体表示一个字符串。
比如C++，比如Java。

这样也顺存 \0
获取长度的时候，也不需要遍历

字符串字面值

我们平时使用的双引号引起来的，就是字符串字面值

写法 —— 第一种写法一定要知道。
"hello world"

存储位置

存储一个叫做只读数据段上的区域。 特点就是
只读
生命周期静态生命周期（活的够久）

还是以 字符数组的形式存的。以 \0结尾的

字符串字面值的操作

一般是在printf的时候传入。这时候，字符串字面值其实退化成了首指针。
它怎么知道这个数组多长？
直接是往后找，找到 \0 认为终止

使用指针接收

```
char *p = "hello";
```

但是需要注意的点是： 使用指针接收，指针指向的内存是只读数据段上的区域
换句话说，就是我们不能通过这个指针去修改这个数据。

字符串变量

声明

因为C中的字符串，就是字符数组。所以声明字符串，就是声明字符数组。
区别就是我们声明的字符数组，应该注意一下 +1就行。
比如，现在要存储长度最多80的字符串，声明字符数组长度应该 80+1

```
char str[80+1];
char *str2;
// 后面这种声明需要注意一下。 这个指针不能直接用，需要指向有效的内存
```

初始化

```
char str[] = "hello";
// 上述代码等价于
char str2[] = {'h', 'e', 'l', 'l', 'o', '\0'};
这是一种语法糖，只是一个简写形式
```

其他的初始化形式都比较简单。
要注意给 \0 留个位置就可以了

```
char str[] = "Showmaker";
char *p = "Showmaker";
这两种写法有这本质的区别
```

第一种写法是个语法糖，会在栈上申请内存。换句话说，它里面的数据可以修改

第二种是一个指针，指针指向只读数据段。所以，指针指向的这片内存无法修改，一修改就报错

字符串的输出和输入

遍历/往屏幕输出

```
char str[] = "hello world!";
for (int i = 0; str[i] != '\0'; i++) {
  printf("%c\n", str[i]);
}
```

—— 建议使用 while循环， for可以

```
char* p = str;
while (*p != '\0') {   // 等价于 *p != 0，即字符不是空字符就继续遍历
  printf("%c\n", *p++);
}
```

—— 一定要注意。 在循环内部，不要*p++了两次。指针挪两次，数据不对

```
printf("%s\n", str);
puts(str);
```

推荐使用while循环。且一定要会
因为遍历操作很常见

—— 需要注意，要么挪指针，要么挪下标，不要一起挪
不要一边挪指针，一边偏移

输入

—— scanf("%s", arr);
输入一个字符串，存储到arr。

不安全，也就是录入的字符串太长了。
会全部录进来

```
scanf("%9s",arr);
```

最多存储9个，第10个存零字符

字符串库

C里面的字符串是字符数组存储的，有一些操作是普遍的，
比如求字符串长度，比如比较两个字符串是否相等。
字符串库简单说就是一系列函数，帮助我们更方便的操作字符串

strlen

```
计算字符串长度
char str1[20] = "hello";
```

```
size_t strlen(const char *s);
```

—— const修饰，所以 *s不能动（指针指向的数据不能动，换句话说字符串内容不会动）

传入的地址应该是一个字符串（以空字符结尾），否则会产生越界访问，未定义行为。

sizeof和strlen区别要知道

```
char str1[10] = "hello";
char *str2 = str1;
```

sizeof用于计算某个位置占用多少大小。比如传数组，得到的是数组占用的大小空间。

strlen用于计算一个字符串长度，计算的是从头到空字符的长度。

面试一般问： 难点、易混淆的点

要自己能实现strlen

strcpy

完成字符串的复制

这个是直接就是复制，不关心能否存下

strncpy: 是一个更安全的字符串复制函数

最多复制n个字符

```
copies a certain amount of characters from one string to another
从一个字符串复制一定数量的字符到另一个字符串
如果 src 的长度小于 n，strncpy() 会向 dest 写入额外的空字节，以确保总共写入 n 个字节。
```

```
strncpy() 函数与之类似，不同之处在于最多只会复制 src 中的 n 个字节。注意：如果 src 的
前 n 个字节中没有空字节，那么存放在 dest 中的字符串将不会以空字节结尾。
```

如果n比较小，比 strlen(src) +1 小。只复制n个

如果n比较巧，刚好等于 strlen(src) + 1。 也复制n个

如果n比较大，比 strlen(src) + 1，前面的数据拷贝过来，后面填 '\0'

```
n的建议： sizeof(dest)-1
dest[sizeof(dest)-1] = '\0'
```

strcat

完成字符串的拼接

```
将src表示的字符串，拼接到dest的后面。
也就是从dest的 \0位置开始覆盖。直到结束
```

strncat: 更安全的cat函数

```
它将从源数据中最多使用 n 个字节；
并且如果源数据包含 n 个或更多字节，那么它就不需要以空字符结尾。
目标字符串 dest 中的最终结果总是以空字符结尾的。
```

有两种情况会终止： 要么复制到了n个字符

要么复制到了src的末尾

最终一定会补一个 \0

```
concat: 拼接
```

建议的n: sizeof(dest) - strlen(dest) -1

strcmp

```
compare: 就是比较两个字符串是否相等。
```

字符串怎么叫相等 —— 字典序。要知道什么是字典序

strlen、strcmp、strcpy

——

```
char str1[100];
str1 = "zhangsan"
strcpy()
```

字符串数组

研究的是有一组字符串如何存储的问题 —— 单个字符串，是字符数组。
多个字符串，字符数组的数组。 所以我们可以使用二维数组来存储。

两种存储方式

二维字符数组

了解一下就行

如果有一个字符串太长，则其他的都要这么长。换句话说，有大量空间浪费

指针数组

```
单个字符串是字符数组，我们可以用一个指针指向它。==》单个字符串可以用指针表示。
一组字符串，可以使用指针数组（是数组，数组里每个元素都是指针），来存储
```

```
指针数组，里面存储的是指针，单个指针，可以指向一个字符数组（字符串）。
所以，我们可以使用指针数组，来存储字符串数组。
```

命令行参数

字符串数组的一个实际案例。

有时候，是允许你运行时给一些参数的。在程序内部，需要接收这个参数，这时候就是使用的字符串数组

关于这块，知道两件事

```
系统会将你给的输入，处理好，处理成指针数组，并且将数据给你。
有两个数据，第一个数据是长度，第二个数据是所有参数
```

所有参数中，即使你一个参数都不写，有（1 程序的名字。）

有什么用

可以传参，影响程序的一些行为。 让程序具有一些灵活性。

比如，我们后期可以写程序，程序复制文件。 1.txt ---> 2.txt

```
copy_file.exe 1.txt  2.txt
```

结构体

就是将一些数据放在一个类型中。让这个一个类型能存放多个数据

结构体的定义 —— 定义一个类型。这个类型就类似于int

声明结构体

```
struct 结构体名 结构体变量名;
int num; // 类似。只是要写struct
```

如果想不加struct。 要用类型别名。 名称区分一下。 使用大驼峰

初始结构体

给结构体对象中的值赋值。

```
如果全部赋值，按个赋进去
如果赋了部分， 后面就是0值
```

数据对齐

——

```
为了保证访问数据的高效，结构体这块存在一个对齐。 简单说，就是凑4字节的整数。
怎么凑，不重要； 重要的是要知道有这个对齐；
后续计算结构体大小，使用sizeof() 不要自己加
```

结构体变量的访问

使用结构体变量名 .字段名来访问这个结构体的字段

是值传递

使用指针接收。

```
Student s1 = { 1, "zhangsan", 'm', 90,80,70 };
Student *p_s = &s1;
```

结构体指针访问字段。

使用指针接收，在函数内部修改外部的结构体变量字段，能否修改成功？

枚举